

Universidad de Guadalajara

Centro Universitario de Ciencias Exactas e Ingenierías



División de Tecnologías para la Integración CiberHumana

Ingeniería en Computación

Programación de Sistemas Avanzados

D01 – IL377 – 223599 – 2026-A

Actividad 2: Primer Avance

Profesor: López Arce Delgado Jorge Ernesto

Integrantes del Equipo:

- Agosto Aceves Carolina
- Garcia Herrera Juan Pablo
- Juárez Rubio Alan Yahir
- Renteria Elias Elisa Yanneth
- Valenzuela López Jorge Yahir

14 de febrero de 2026

Tabla de Contenidos

Tabla de Contenidos	2
Diagrama de la Topología de Red	3
Roles de los Nodos	4
Site WireGuard	4
Site 1 (10.5.5.2/32)	4
Site 2 (10.5.5.3/32)	4
Site 3 (10.5.5.4/32)	5
Site 4 (10.5.5.5/32)	5
Site 5 (10.5.5.6/32)	6
Llaves Generadas de WireGuard	7
Conexión a la VPN	10
Instalación de Herramientas Esenciales	12
Dockerfile, Docker-compose y Kubernetes	14
Dockerfile	14
Docker-compose.yml	14
Kubernetes	16
Problemas Encontrados y Decisiones Técnicas Tomadas	17
Configuración de la VPN en Wireguard del lado del Servidor	17
Problema: El Conjunto de Mandelbrot	18
Usos y aplicaciones	18
Evidencia de la Metodología Scrum	19
Repositorio en Github	20

Diagrama de la Topología de Red

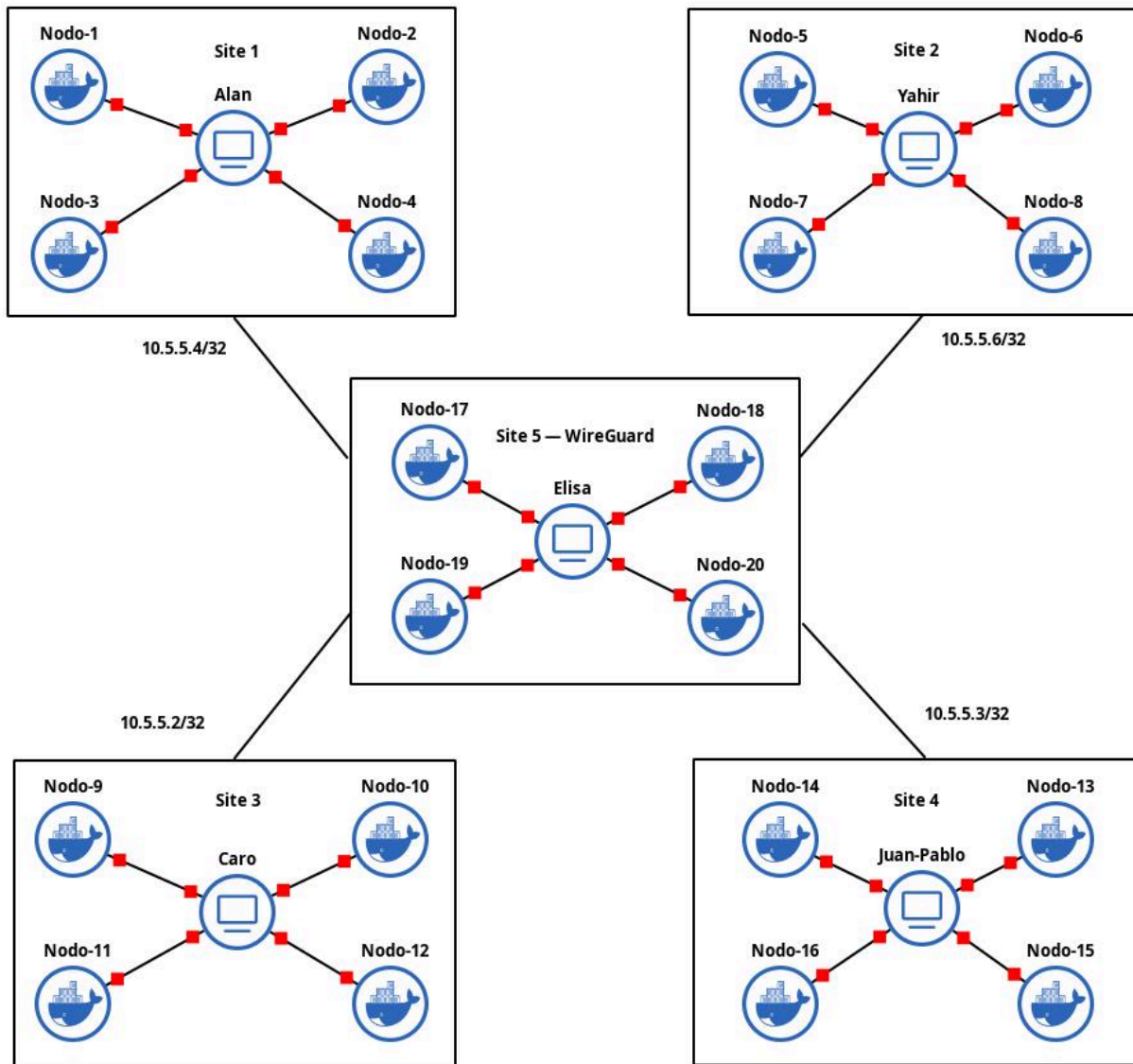


Fig. 1. Topología de la Red Hub-and-spoke

Roles de los Nodos

Site WireGuard

Rol: Hub central de la VPN (10.5.5.1/32)

Función: ser el servidor Wireguard el cual se encargará de recibir las conexiones de:

- Site 1
- Site 2
- Site 3
- Site 4
- Site 5

Site 1 (10.5.5.2/32)

Rol del host.

- peer de WireGuard
- Cliente de la VPN
- Host Docker
- Ejecutar 4 workers

Rol de los nodos del Docker:

- Workers de computo
- ejecutar tareas distribuidas
- se comunican únicamente por la VPN

Nodos:

- Nodo 1
- Nodo 2
- Nodo 3
- Nodo 4

Site 2 (10.5.5.3/32)

Rol del host.

- peer de WireGuard
- Cliente de la VPN

- Host Docker
- Ejecutar 4 workers

Rol de los nodos del Docker:

- Workers de computo
- ejecutar tareas distribuidas
- se comunican únicamente por la VPN

Nodos:

- Nodo 5
- Nodo 6
- Nodo 7
- Nodo 8

Site 3 (10.5.5.4/32)

Rol del host.

- peer de WireGuard
- Cliente de la VPN
- Host Docker
- Ejecutar 4 workers

Rol de los nodos del Docker:

- Workers de computo
- ejecutar tareas distribuidas
- se comunican únicamente por la VPN

Nodos:

- Nodo 9
- Nodo 10
- Nodo 11
- Nodo 12

Site 4 (10.5.5.5/32)

Rol del host.

- peer de WireGuard
- Cliente de la VPN
- Host Docker

- Ejecutar 4 workers

Rol de los nodos del Docker:

- Workers de computo
- ejecutar tareas distribuidas
- se comunican únicamente por la VPN

Nodos:

- Nodo 13
- Nodo 14
- Nodo 15
- Nodo 16

Site 5 (10.5.5.6/32)

Rol del host.

- peer de WireGuard
- Cliente de la VPN
- Host Docker
- Ejecutar 4 workers

Rol de los nodos del Docker:

- Workers de computo
- ejecutar tareas distribuidas
- se comunican únicamente por la VPN

Nodos:

- Nodo 17
- Nodo 18
- Nodo 19
- Nodo 20

Llaves Generadas de WireGuard

```
elisa@MSI:/mnt/c/Users/elisa$ sudo ls -la /etc/wireguard/
[sudo] password for elisa:
total 36
drwx----- 2 root root 4096 Feb 11 19:05 .
drwxr-xr-x 89 root root 4096 Feb 15 14:33 ..
-rw-r--r-- 1 root root 251 Feb 11 17:28 client_alanyahir.conf
-rw-r--r-- 1 root root 251 Feb 11 17:28 client_caro.conf
-rw-r--r-- 1 root root 251 Feb 11 17:28 client_elisaelias.conf
-rw-r--r-- 1 root root 251 Feb 11 17:28 client_jorgeyahir.conf
-rw-r--r-- 1 root root 251 Feb 11 17:28 client_juanpablo.conf
-rw----- 1 root root 45 Jan 30 12:32 private.key
-rw----- 1 root root 918 Feb 11 19:05 wg0.conf
```

Fig. 2. Directorio de claves generadas para la VPN

```
elisa@MSI:/mnt/c/Users/elisa$ sudo cat /etc/wireguard/private.key
ONYfQsTdEbUUDiUsj20seA8BruKZGXF4sHhyKwYcfW4=
```

Fig. 3. Clave privada del servidor

```
elisa@MSI:/mnt/c/Users/elisa$ sudo cat /etc/wireguard/client_caro.conf
[Interface]
PrivateKey = cJSLBVLdFnzoKHiZOVv5ITueVXg2QeOCWg72/IcQGVI=
Address = 10.5.5.2/32
DNS = 8.8.8.8

[Peer]
PublicKey = 36ANUqXr4lbvke+IaSGVDKmPBmKiPNfWPbjP/PFpvgE=
Endpoint = 187.139.127.50:51820
AllowedIPs = 0.0.0.0/0
PersistentKeepalive = 25
```

Fig. 4. Claves para Carolina

```
elisa@MSI:/mnt/c/Users/elisa$ sudo cat /etc/wireguard/client_juanpablo.conf
[Interface]
PrivateKey = CBt767PzzLOSIIfIOJfMhvNeZIqwcya3m+4Gr5RmViFg=
Address = 10.5.5.3/32
DNS = 8.8.8.8

[Peer]
PublicKey = 36ANUqXr4lbvke+IaSGVDKmPBmKiPNfWPbjP/PFpvgE=
Endpoint = 187.139.127.50:51820
AllowedIPs = 0.0.0.0/0
PersistentKeepalive = 25
```

Fig. 5. Claves para Juan Pablo

```
elisa@MSI:/mnt/c/Users/elisa$ sudo cat /etc/wireguard/client_alanyahir.conf
[Interface]
PrivateKey = yJ0vDFR89fgHnjF7oJruySuTmQ+k0yB63gQ5ALCe4HE=
Address = 10.5.5.4/32
DNS = 8.8.8.8

[Peer]
PublicKey = 36ANUqXr4lbvke+IaSGVDKmpBmKiPNfWPbjP/PFpvgE=
Endpoint = 187.139.127.50:51820
AllowedIPs = 0.0.0.0/0
PersistentKeepalive = 25
```

Fig. 6. Claves para Alan Yahir

```
elisa@MSI:/mnt/c/Users/elisa$ sudo cat /etc/wireguard/client_jorgeyahir.conf
[Interface]
PrivateKey = QHMFFJEG7A0T6/bLHx4pb/LBARvUImrC19fUmTH9nFI=
Address = 10.5.5.6/32
DNS = 8.8.8.8

[Peer]
PublicKey = 36ANUqXr4lbvke+IaSGVDKmpBmKiPNfWPbjP/PFpvgE=
Endpoint = 187.139.127.50:51820
AllowedIPs = 0.0.0.0/0
PersistentKeepalive = 25
```

Fig. 7. Claves para Jorge Yahir

```
elisa@MSI:/mnt/c/Users/elisa$ sudo cat /etc/wireguard/client_elisaelias.conf
[Interface]
PrivateKey = OMFwtgtDCt7z4m8P1RYgky8udm7SDmhh17Si0F63+HM=
Address = 10.5.5.5/32
DNS = 8.8.8.8

[Peer]
PublicKey = 36ANUqXr4lbvke+IaSGVDKmpBmKiPNfWPbjP/PFpvgE=
Endpoint = 187.139.127.50:51820
AllowedIPs = 0.0.0.0/0
PersistentKeepalive = 25
```

Fig. 8. Claves para Elisa Elias


```
elisa@MSI:/mnt/c/Users/elisa$ sudo cat /etc/wireguard/wg0.conf
[Interface]
Address = 10.5.5.1/24
ListenPort = 51820
PrivateKey = ONYfQsTdEbUUDiUsj20seA8BruKZGXF4sHhyKwYcfW4=

PostUp = sysctl -w net.ipv4.ip_forward=1
PostUp = iptables -A FORWARD -i wg0 -j ACCEPT
PostUp = iptables -A FORWARD -o wg0 -j ACCEPT
PostUp = iptables -t nat -A POSTROUTING -o eth4 -j MASQUERADE

PostDown = iptables -D FORWARD -i wg0 -j ACCEPT
PostDown = iptables -D FORWARD -o wg0 -j ACCEPT
PostDown = iptables -t nat -D POSTROUTING -o eth4 -j MASQUERADE

[Peer]
PublicKey = xPTPf4NfKDJMSZ8lpEFutAxCN/kGgnORLvr4WldFBwc=
AllowedIPs = 10.5.5.2/32

[Peer]
PublicKey = mHVnV0QDZW6P3UkhKxpW4mKu6A1RK9eoK9QAvXT2Rjo=
AllowedIPs = 10.5.5.3/32

[Peer]
PublicKey = C4rPIuZhUh+4XvXgxasngX3kpqLPCLVHxGYbBCpBA0=
AllowedIPs = 10.5.5.4/32

[Peer]
PublicKey = E1127VnkucLMvpph165CUdRhc3Rkk09xYkwQ/XP08X0=
AllowedIPs = 10.5.5.5/32

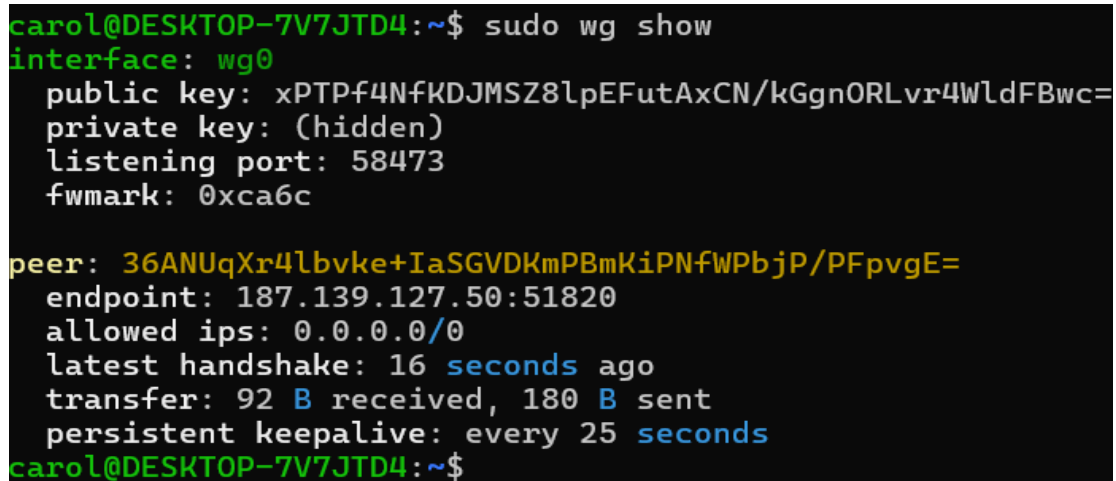
[Peer]
PublicKey = Zg7+rCi4vgToBzcOuHuK7CkxnkjFNwki0DxkaU06kM=
AllowedIPs = 10.5.5.6/32
```

Fig. 9. Archivo de Configuración para la VPN

Conexión a la VPN

Para que todos los nodos puedan conectarse al Servidor VPN en Wireguard se realizó la instalación de Wireguard en cada máquina virtual de los distintos nodos. Utilizando el comando `sudo apt install wireguard` para realizar la instalación.

Se debe crear una carpeta para Wireguard con el comando `sudo mkdir -p /etc/wireguard` y crear el archivo de configuración con el comando `sudo nano /etc/wireguard/wg0.conf`, después se debe ajustar los permisos con `sudo chmod 600 /etc/wireguard/wg0.conf`. Para levantar la VPN se utiliza la instrucción `sudo wg-quick up wg0` y verificar la conexión con `wg-show`.



```
carol@DESKTOP-7V7JTD4:~$ sudo wg show
interface: wg0
  public key: xPTPf4NfKDJMSZ8lpEFutAxCN/kGgnORLvr4WldFBwc=
  private key: (hidden)
  listening port: 58473
  fwmark: 0xca6c

peer: 36ANUqXr4lbvke+IaSGVDKmPBmKiPNfWPbjP/PFpvgE=
  endpoint: 187.139.127.50:51820
  allowed ips: 0.0.0.0/0
  latest handshake: 16 seconds ago
  transfer: 92 B received, 180 B sent
  persistent keepalive: every 25 seconds
carol@DESKTOP-7V7JTD4:~$
```

Fig. 10. Conexión de un nodo establecida con el servidor

```
PS C:\WINDOWS\system32> & "C:\Program Files\WireGuard\wg.exe" show
>>
interface: wg0_server
  public key: 36ANUqXr4lbvke+IaSGVDKmPBmKiPNfWPbjP/PFpvgE=
  private key: (hidden)
  listening port: 51820

peer: xPTPf4NfKDJMSZ8lpEFutAxCN/kGgn0RLvr4WldFBwc=
  endpoint: 187.139.254.128:55286
  allowed ips: 10.5.5.2/32
  latest handshake: 4 minutes, 37 seconds ago
  transfer: 2.22 KiB received, 1.33 KiB sent

peer: Zg7+rCi4vgToBzcOuHuK7CkxnkjFNwki0dDxkaU06kM=
  allowed ips: 10.5.5.6/32

peer: mHVnV0QDZW6P3UkhKxpW4mKu6A1RK9eoK9QAvXT2Rjo=
  allowed ips: 10.5.5.3/32

peer: E1127VnkucLMvpvh165CUdRhc3RKk09xYkwQ/XP08X0=
  allowed ips: 10.5.5.5/32

peer: C4rPIuZhUh+4XvXgXasngX3kpqlPCoLVHxGYbBCpBA0=
  allowed ips: 10.5.5.4/32
```

Fig. 11. Conexión del lado del Servidor VPN con Wireguard

Instalación de Herramientas Esenciales

La configuración del entorno es esencial para poder establecer el Sistema Distribuido por lo que es necesario de diversas herramientas que permitan garantizar la conectividad, el despliegue y la evaluación del rendimiento. A continuación, se muestran cada una de las herramientas que fueron instaladas específicamente para el desarrollo del proyecto.

Para demostrar que las herramientas se encuentran instaladas, ejecutamos los siguientes comandos:

```
Shell
docker --version
git --version
wg --version
k3s --version
iperf3 --version
iptables --version
ufw --version
```

```
root@debian:~# docker --version
Docker version 29.2.1, build a5c7197
root@debian:~# git --version
git version 2.47.3
root@debian:~# wg --version
wireguard-tools v1.0.20210914 - https://git.zx2c4.com/wireguard-tools/
root@debian:~# k3s --version
k3s version v1.34.4+k3s1 (c6017918)
go version go1.24.12
root@debian:~# iperf3 --version
iperf 3.18 (cJSON 1.7.15)
Linux debian 6.12.69+deb13-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.69-1 (2026-02-08) x86_64
Optional features available: CPU affinity setting, IPv6 flow label, SCTP, TCP congestion algorithm setting, sendfile / zerocopy, socket pacing, authentication, bind to device, support IPv4 don't fragment, POSIX threads
root@debian:~# iptables --version
iptables v1.8.11 (nf_tables)
root@debian:~# ufw --version
ufw 0.36.2
Copyright 2008-2023 Canonical Ltd.
```

Fig. 12. Herramientas Instaladas – Alan

```
carol@DESKTOP-7V7JTD4:~$ wg --version
wireguard-tools v1.0.20210914 - https://git.zx2c4.com/wireguard-tools/
carol@DESKTOP-7V7JTD4:~$ git --version
git version 2.34.1
carol@DESKTOP-7V7JTD4:~$ docker --version
Docker version 29.2.0, build 0b9d198
carol@DESKTOP-7V7JTD4:~$ |
```

Fig. 13. Herramientas Instaladas – Caro

Dockerfile, Docker-compose y Kubernetes

Dockerfile

Dentro de un Sistema Distribuido es necesario la implementación de contenedores. Se establece un Dockerfile base para la creación de los contenedores. El Dockerfile utiliza la imagen oficial de Rust versión 1.93 y está basado en Linux Debian.

```
None
FROM rust:1.93-trixie

WORKDIR /app

COPY Cargo.lock ./
COPY Cargo.toml ./

# PERFORMANCE: increase the rebuild speed because it loads all rust dependencies
# See: https://www.youtube.com/watch?v=5Wfpzfniu4I
RUN mkdir -p src \
    && echo 'fn main() {}' > src/main.rs \
    && cargo build --release \
    # Remove dummy artifacts
    && rm -rf src target/release/deps/mandelbrot*

COPY ./src/ ./src/
RUN cargo build --release

# Executes the pre-compiled rust release binary, just like `cargo run --release`
CMD ["/target/release/mandelbrot"]
```

Docker-compose.yml

Para gestionar los contenedores se utilizó el siguiente archivo de configuración docker-compose.yml

```
None
services:
  worker1:
    build:
      context: ../rust
      dockerfile: ../docker/Dockerfile
    container_name: worker1
    restart: unless-stopped
    environment:
      - ROLE=coordinator
      # Workers to connect to (comma-separated)
      - WORKERS=worker2:8080,worker3:8080,worker4:8080
    ports:
      - "8080:8080"

  worker2:
    build:
      context: ../rust
      dockerfile: ../docker/Dockerfile
    container_name: worker2
    restart: unless-stopped
    environment:
      - ROLE=worker
      - LISTEN_ADDR=0.0.0.0:8080

  worker3:
    build:
      context: ../rust
      dockerfile: ../docker/Dockerfile
    container_name: worker3
    restart: unless-stopped
    environment:
      - ROLE=worker
      - LISTEN_ADDR=0.0.0.0:8080
```

```
worker4:
  build:
    context: ../rust
    dockerfile: ../docker/Dockerfile
  container_name: worker4
  restart: unless-stopped
  environment:
    - ROLE=worker
    - LISTEN_ADDR=0.0.0.0:8080
```

En este archivo de configuración levanta 4 contenedores iguales a partir del Dockerfile. Cada bloque es un servicio independiente. Todos los contenedores hacen las mismas funciones pero en instancias separadas. El contexto indica que se utilizó para crear la imagen la carpeta rust y unless-stopped para mantener los servicios activos tras los reinicios del servidor.

Kubernetes

Para el despliegue de la aplicación en cada uno de los *sites* y, respectivamente, en cada uno de los contenedores hemos decidido utilizar **kubernetes**, específicamente **k3s**. Actualmente nos encontramos en la fase de investigación, aprendiendo sobre esta herramienta para después poder integrarla en nuestro proyecto.

Problemas Encontrados y Decisiones Técnicas Tomadas

Configuración de la VPN en Wireguard del lado del Servidor

Del lado del servidor fue necesario configurar el router para permitir enrutar correctamente el tráfico de la VPN creada con Wireguard asegurando la comunicación entre los nodos del sistema distribuido.

Así mismo fue necesario configurar el ICS (Conexión Compartida a Internet) para permitir a los distintos nodos y comunicarse dentro de la red local.

Problema: El Conjunto de Mandelbrot

Este problema consiste en determinar para cada punto en un plano complejo representado por la variable c , si dicho punto pertenece o no a un cierto conjunto definido mediante una iteración no lineal. Para cada valor de C se construye la sucesión empezando por:

$$z_0 = 0 \quad y \quad z_{n+1} = z_n^2 + c$$

El objetivo es decidir si esta sucesión permanece acotada o si por el contrario, tiende a infinito cuando el número de iteraciones aumenta. El conjunto de mandelbrot está formado por todos los valores de c para los que la sucesión es acotada, y su representación gráfica se obtiene asignando a cada píxel del plano un valor de c , y calculando cuántas iteraciones tarda en escapar más allá de un cierto radio. Esto lo convierte en un problema muy costoso computacionalmente hablando y paralelizable, porque el cálculo de cada punto c es independiente a los demás y puede distribuirse entre múltiples procesos o nodos de cómputo.

Usos y aplicaciones

El conjunto de Mandelbrot, aunado a la geometría fractal, tienen multiplicidad de usos y aplicaciones, algunos de ellos son:

- Gráficos por computadora: En este aspecto se utilizan derivadas del conjunto de Mandelbrot para hacer la generación de paisajes artificiales (montañas, cerros, costas, nubes, texturas de planetas, etc) y efectos visuales, ya que este tipo de fractales permiten crear formas naturales muy detalladas a partir de reglas matemáticas, reduciendo el trabajo manual y los costos que conlleva.
- Diseño de antenas fractales, que aprovechan estructuras autosimilares para funcionar en múltiples bandas de frecuencia ocupando menos espacio físico, y también en técnicas de procesamiento y compresión de imágenes, donde patrones fractales ayudan a representar texturas con menos datos.
- También en el modelado de fenómenos naturales, cómo lo pueden ser líneas de costa, sistemas de ríos, relieves de montañas, ramas de árboles o la redes de vasos sanguíneos.

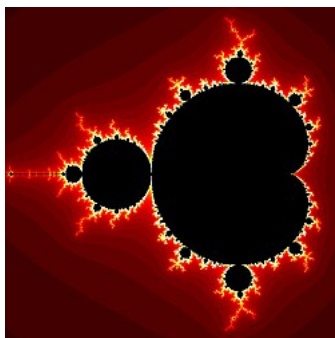


Fig. 14. Conjunto de Mandelbrot Graficado

Evidencia de la Metodología Scrum

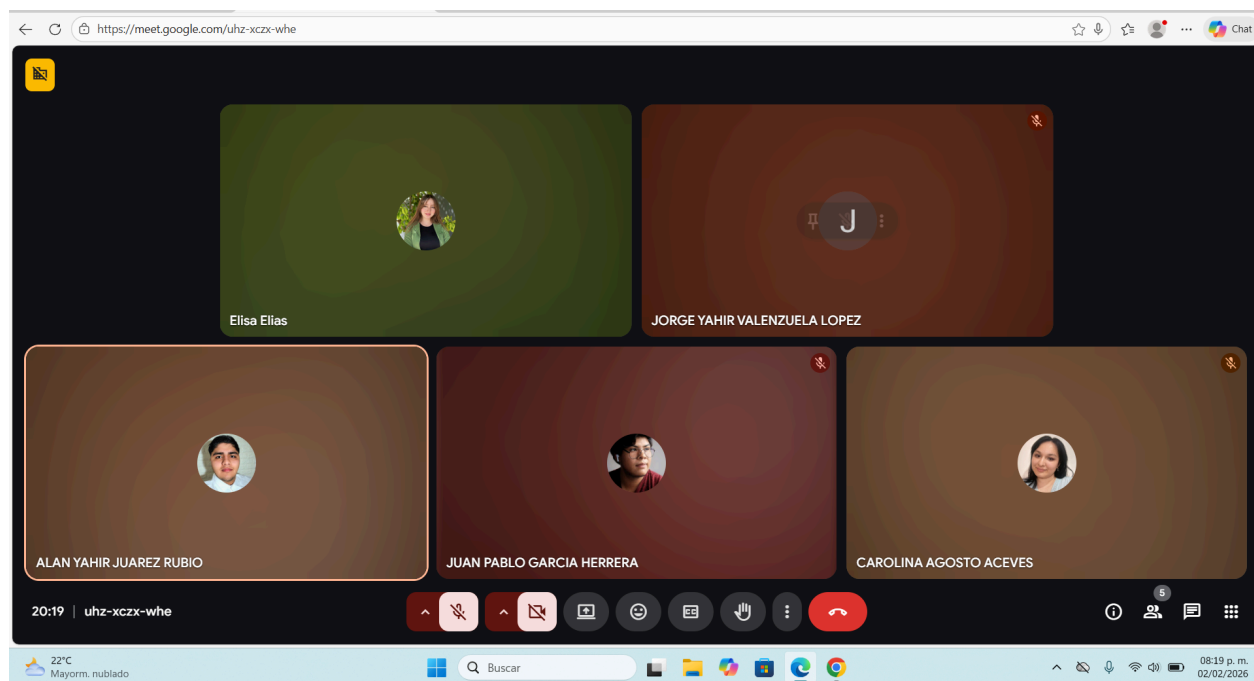


Fig. 15. Sesión en Meet para la organización del proyecto.

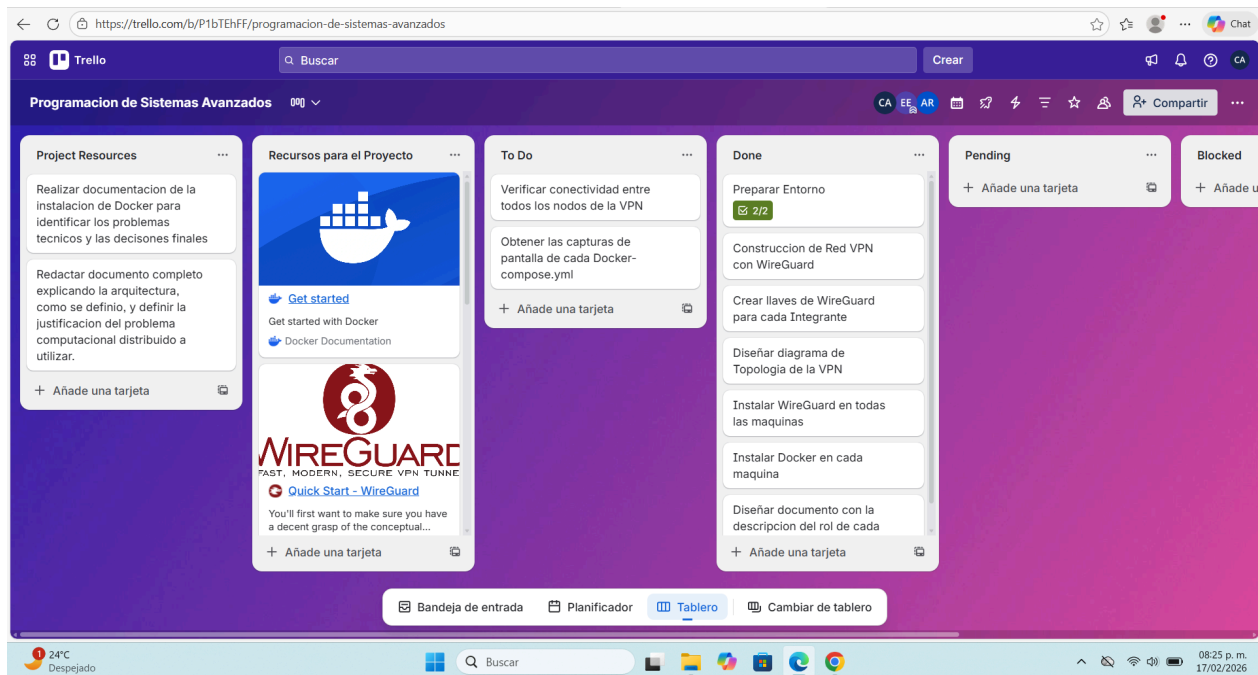


Fig. 16. Gestión de tareas en la plataforma Trello

Repositorio en Github

Enlace al Repositorio

https://github.com/MrDonkey08/Quantum-Core_Sistema-Distribuido.git